

Report: Analysis of NLSCYA data

Objectives

- Can the presence of interactions- and other non-linear effects be inferred by comparing the prediction performance of purely linear base model with random forest base models in MVS?
- Can residualizing the outcome with respect to a linear model help to evaluate non-linear or interaction signal using random forests?

Summary & Conclusions

- Using random forest base models in the MVS configuration showed slight improvement in predictive performance compared with MVS using only linear models.
- In the MVS configuration where additional random forest base learners were fitted to the residualized outcome after subtracting the linear predictions, the estimated coefficients suggest that the random forest models capture additional signal beyond the linear predictions.
- Conclusion: Modest predictive improvements, but the coefficients of the MVS configurations with random forest base models indicate the presence of signal not captured by purely linear base models.

Data

The data is a sample of the Children of the National Longitudinal Survey of Youth 1979, based on a pre-selection by Deming (2009).

For the analysis we selected cases based on available data, resulting in a sample of 703 children. The outcome variable is a combined cognitive test score.

The covariate group labels are:

- Child characteristics ($n_{\text{covariates}} = 8$),
- Family characteristics ($n_{\text{covariates}} = 4$),
- Mother characteristics ($n_{\text{covariates}} = 5$),
- Pre- and postnatal characteristics ($n_{\text{covariates}} = 2$).

Cross-validation procedure

For evaluating predictive performance, 10-fold cross-validation was used. Because the data included siblings, mother's ID was used to ensure that the children with the same mother were assigned to the same fold. The size of each fold was around 70.

Results

Table 1

Predictive performance

	MVS Linear	MVS Residual Models
MSE (SD)	481 (63)	478 (66)
Test R^2	0.20	0.21

Note. The variance of the outcome in the complete data is 605. Predictive R^2 was calculated as $1 - \text{mse} / \text{var}(y_{\text{test}})$.

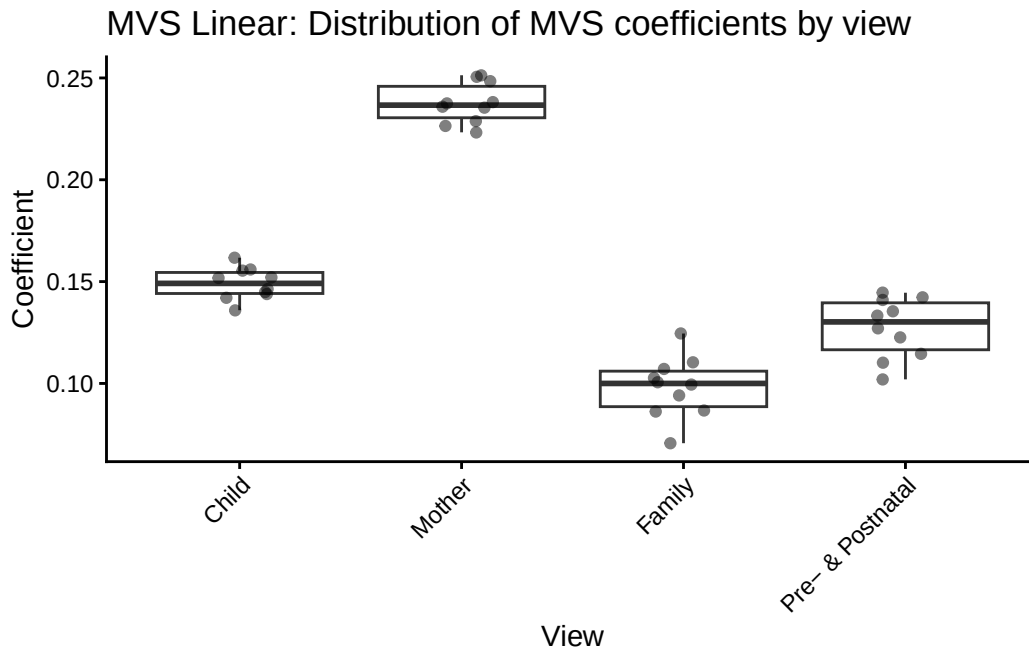
Table 2

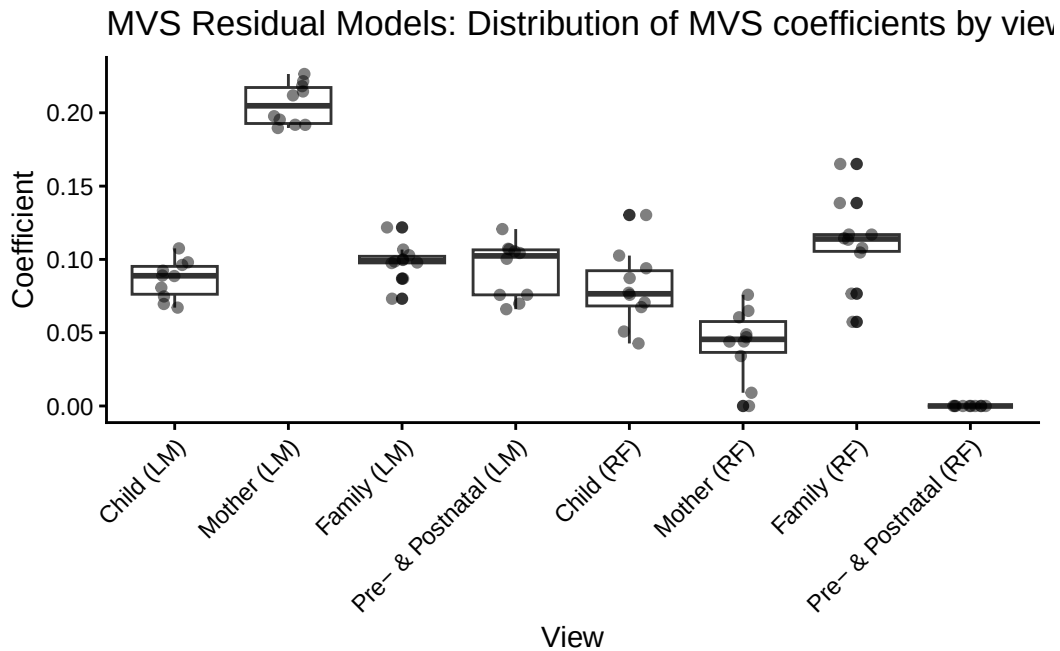
Medians of standardized coefficients

	MVS Linear	MVS Residual Models	
		Linear	Random Forest
Child	0.15	0.09	0.08
Mother	0.24	0.21	0.05
Family	0.10	0.10	0.12
Pre- & Postnatal	0.13	0.10	0

Plots of coefficient distributions

- The plots show the coefficient distributions obtained over the different fold combinations.
- In MVS residual models, views 1 to 4 correspond to linear models fitted to the covariate groups, and views 5 to 8 correspond to random forest models fitted to the same covariate groups but on the residualized outcome.





Variable names and group labels

	Var.names	Group
2	Race_Child	child chr
3	Sex_Child	child chr
4	BirthWeight	child chr
5	BirthOrder	child chr
1	BornEarlyorLate	child chr
21	BornOnTime	child chr
31	RepeatNone98	child chr
41	Illness_1stYr	child chr
6	PermInc	family chr
7	FamilySize79	family chr
8	Grandmother86	family chr
18	HighGrade_GDad79	family chr
10	AgeAFQT	mother chr
11	MomDropout	mother chr
12	Height_Mom81	mother chr
13	MothED	mother chr
14	Age_Moth_Birth	mother chr
15	Freq_Alc_BefBirth	Pre- and postnatal chr
16	Breastfed	Pre- and postnatal chr

Model fitting syntax

```
# Libraries
library(glmnet)
library(randomForest)
library(mvs)

# Helper function to extract results
extract_results <- function(model, y_train = y_train, newx = X_test, newy = y_test) {
  pred <- predict(model, newx = newx)
  test_mse <- mean((pred - newy)^2)
  test_r2 <- 1 - test_mse / var(newy)
  coefs <- coef(model)$`Level 2`[[1]]
  cv_variances <- apply(model$`Level 1`$CVs, 2, var)
  cv_sds <- sqrt(cv_variances)
  std_coefs <- coefs[-1,] * (cv_sds / sd(y_train))

  list(
    model = model,
    coefficients = coefs,
    std_coefficients = std_coefs,
    test_MSE = test_mse,
    test_R2 = test_r2,
    cv_variances = cv_variances
  )
}

# Helper function to create cv folds based on group IDs
make_group_cv_folds <- function(data, group_var, k = 10, seed = 123) {
  set.seed(seed)

  # Get unique group IDs
  groups <- unique(data[[group_var]])

  # Shuffle group IDs
  groups <- sample(groups)
```

```

# Assign each group to a fold
fold_ids <- rep(seq_len(k), length.out = length(groups))
group_fold_map <- data.frame(
  group = groups,
  fold = fold_ids
)

# Create folds
cv_fold <- group_fold_map$fold[
  match(data[[group_var]], group_fold_map$group)
]

return(cv_fold)
}

# Load data
subset_NLSCYA_cc <- readRDS('NLSCYA_data_cc.rds')

names(subset_NLSCYA_cc)

# MVS application to NHANES data ----

# Set seed
set.seed(42)

# Create a vector with CV folds
fold_inx <- make_group_cv_folds(subset_NLSCYA_cc, "MotherID")

# Exclude child- and mother ID
deming_data_wthout_IDs <- subset_NLSCYA_cc[,!names(subset_NLSCYA_cc) %in% c("MotherID", "ChildID")]

# Define view indices based on source, taking into account dummies for factors
view_ids <- c(
  rep(1, 16), # Child characteristics
  rep(2, 6), # Mother characteristics
  rep(3, 5), # Family characteristics
  rep(4, 3) # Pre- and postnatal conditions
)

# Create list objects to collect extracted results
res_mvs_default <- list()
res_mvs_rf <- list()

```

```

res_mvs_repeated <- list()
res_mvs_resid <- list()

for(i in 1:10) {

  # Split into training and test data

  X_train <- model.matrix( ~ . - 1, data = deming_data_wthout_IDs[fold_inx != i, -20])
  y_train <- deming_data_wthout_IDs[fold_inx != i, 20]
  X_test <- model.matrix( ~ . - 1, data = deming_data_wthout_IDs[fold_inx == i, -20])
  y_test <- deming_data_wthout_IDs[fold_inx == i, 20]

  # Fit MVS models ----

  # Fit MVS default
  mvs_default <- MVS(
    x = X_train,
    y = y_train,
    views = view_ids,
    progress = FALSE,
    family = "gaussian",
    std.meta = "TRUE"
  )

  # Compute results
  res_mvs_default[[i]] <- extract_results(mvs_default, y_train = y_train, newx = X_test, newy = y_test)

  ## RF MVS (four views and groups, random forest baselearners, lasso metalearner);
  type <- list(level1 = "RF", meta = "StaPLR")
  mvs_rf <- MVS(
    x = X_train,
    y = y_train,
    views = view_ids,
    type = type,
    progress = FALSE,
    family = "gaussian",
    std.meta = "TRUE"
  )

  # Compute results
  res_mvs_rf[[i]] <- extract_results(mvs_rf, y_train = y_train, newx = X_test, newy = y_test)
}

```

```

# Repeated views MVS

# Create a new predictor matrix Xrep with repeated views
Xrep_train <- cbind(X_train, X_train)
Xrep_test <- cbind(X_test, X_test)
view_ids_rep <- c(view_ids, (view_ids + 4))

# Specify type of base learner per view
type <- list(
  level1 = c(
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "RF",
    "RF",
    "RF",
    "RF"
  ),
  meta = "StaPLR"
)

# Fit MVS model
mvs_rep <- MVS(
  x = Xrep_train,
  y = y_train,
  views = view_ids_rep,
  type = type,
  progress = FALSE,
  family = "gaussian",
  std.meta = "TRUE"
)

# Compute results
res_mvs_repeated[[i]] <- extract_results(mvs_rep, y_train = y_train, newx = Xrep_test, newy = y_test)

# MVS residual models

# Specify type of base learner per view
type <- list(
  level1 = c(
    "StaPLR",

```



```

      "StaPLR",
      "StaPLR",
      "StaPLR",
      "RF",
      "RF",
      "RF",
      "RF"
    ),
    meta = "StaPLR"
  )

# Fit model with offset
mvs_res_mod <- MVS(
  x = Xrep_train,
  y = y_train,
  views = view_ids_rep,
  type = type,
  progress = FALSE,
  family = "gaussian",
  offsets = list(c(1, 5), c(2, 6), c(3, 7), c(4, 8)),
  std.meta = "TRUE"
)

# Compute results
res_mvs_resid[[i]] <- extract_results(mvs_res_mod, y_train = y_train, newx = Xrep_test, newy = y_test)
}

```

```

# Helper functions

# Helper function to summarize results
summarize_results <- function(res_list) {
  mse <- sapply(res_list, `[`, "test_MSE")
  r2 <- sapply(res_list, `[`, "test_R2")

  coefs <- do.call(cbind, lapply(res_list, `[`, "std_coefficients"))

  list(
    MSE_mean = mean(mse, na.rm = TRUE),
    MSE_median = median(mse, na.rm = TRUE),

    R2_mean = mean(r2, na.rm = TRUE),

```

```

    R2_median = median(r2, na.rm = TRUE),

    coef_mean = rowMeans(coefs, na.rm = TRUE),
    coef_median = apply(coefs, 1, median, na.rm = TRUE)
  )
}

# Helper function to plot coefficients
library(Matrix)
library(reshape2)
library(ggplot2)

plot_coefficients <- function(res_list, title, view.names = NULL) {
  coefs <- do.call(cbind, lapply(res_list, `[`, "std_coefficients"))

  coef_mat <- as.matrix(coefs)
  colnames(coef_mat) <- c(
    "fold 1",
    "fold 2",
    "fold 3",
    "fold 4",
    "fold 5",
    "fold 6" ,
    "fold 7",
    "fold 8",
    "fold 9",
    "fold 10"
  )

  coef_long <- reshape2::melt(coef_mat,
                              varnames = c("view", "fold"),
                              value.name = "coefficient")

  if( !is.null(view.names) ) {levels(coef_long$view) <- view.names}

  #coef_long <- cbind(coef_long, view.names)

  ggplot(coef_long, aes(x = view, y = coefficient)) +
    geom_boxplot() +
    geom_jitter(width = 0.15, alpha = 0.5) +
    #coord_flip() +

```

```
labs(x = "View", y = "Coefficient", title = title) +  
theme_classic() +  
theme(axis.text.x = element_text(angle = 45, hjust = 1))  
}
```

References

Deming, D (2009). Early Childhood Intervention and Life-Cycle Skill Development: Evidence from Head Start. *American Economic Journal: Applied Economics* 1 (3): 111–34.